

Caso práctico



[Ministerio de Educación](#) (Uso educativo)

Ada ha asignado un proyecto a Juan que contará con Ana para trabajar en él. De este modo Ana irá aprendiendo a la vez que ayuda a Juan en tan importante tarea.



[Ministerio de Educación](#) (Uso educativo)

Se trata de un proyecto importante y puede suponer muchas ventas, por tanto, una gran expansión para la empresa. Así que Ada supervisará todo el trabajo de Juan para que no haya ningún problema.

El director de una importante empresa se dirigió a BK programación para pedirles que desarrollen un sitio web de juegos online, al que se podrán conectar usuarios para jugar partidas. Se tiene que realizar un diseño de la base de datos que soporte la operativa de este sitio web.

Una cuestión vital en la aplicación es el almacenamiento de los datos. Los datos de los usuarios, el acceso de éstos, registro de las distintas partidas y juegos que se crean y el control de las compras de crédito por parte de los jugadores. Todo deberá guardarse en bases de datos, para su tratamiento y recuperación las veces que haga falta.

Como en BK programación trabajan sobre todo con Oracle, desde el primer momento Juan, con el visto bueno de Ada, tiene claro que van a tener que utilizar bases de datos relacionales y Oracle.



[Ministerio de Educación y Formación Profesional](#). (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#)

1.- Modelo de datos.

Caso práctico



[Ministerio de Educación](#) (Uso educativo nc)

Juan y Ana se han puesto en marcha con este nuevo proyecto. Ambos saben que lo primero que tienen que hacer es trabajar con la información que les han dado. Ya saben las ideas que el cliente tiene, ahora es



[Ministerio de Educación](#) (Uso educativo nc)

necesario pasarlo a un formato con el que poder trabajar. Una de las primeras cosas que deben hacer es trazar un borrador donde plasmar lo que ahora mismo está en sus cabezas y en las anotaciones recogidas mientras hablaban con el cliente.



[Jesús García Arámbula](#)
(Dominio público)

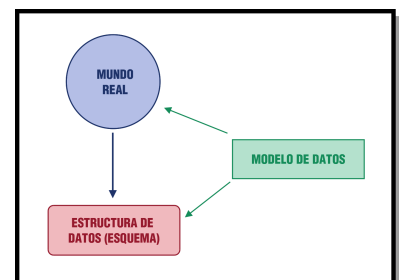
Según el [DRAE](#), un modelo es, entre otras definiciones, el esquema teórico, generalmente en forma matemática, de un sistema o de una realidad compleja. Podemos decir que es la representación de cualquier aspecto o tema extraído del mundo real.

¿Qué sería entonces un modelo de datos? Aquél que nos permite describir los elementos que intervienen en una realidad o en un problema dado y la forma en que se relacionan dichos elementos entre sí.

Contextualizando, un modelo de datos es por tanto un conjunto de métodos y reglas que indican cómo se ha de almacenar la información y cómo se han de manipular los datos.

En informática, el modelo de datos se implementa con un lenguaje utilizado para la descripción de una base de datos. Con este lenguaje vamos a poder describir las estructuras de los datos (tipos de datos y relaciones entre ellos), las restricciones de integridad (condiciones que deben cumplir los datos, según las necesidades de nuestro modelo basado en la realidad) y las operaciones de manipulación de los datos (insertado, borrado, modificación de datos).

Ese lenguaje, por lo general, presenta dos sublenguajes:



[Ministerio de Educación](#) (Uso educativo nc)

- ✓ **Lenguaje de Definición de Datos o DDL (Data Definition Language)** , cuya función es describir, de una forma abstracta, las estructuras de datos y las restricciones de integridad.
- ✓ **Lenguaje de Manipulación de Datos o DML (Data Manipulation Language)**, que sirve para describir las operaciones de manipulación de los datos.

Existen tres **fases de modelado** en el diseño de una base de datos basadas en el nivel de abstracción, es decir, en lo alejado que esté del mundo real y que deben ser realizadas en orden:

1. **Modelo de datos conceptual:** Se utiliza en el diseño conceptual. Es una representación (normalmente gráfica) de la realidad no comprometida con ningún entorno informático. Describen las estructuras de datos y restricciones de integridad. Se utilizan durante la etapa de análisis de un problema dado, y están orientados a representar los elementos que intervienen y sus relaciones. Es una fase muy importante ya que cualquier error en esta fase es arrastrado a las siguientes. Ejemplo, Modelo Entidad-Relación.
2. **Modelo Lógico:** Se utiliza en el diseño lógico. Determinan unos criterios de almacenamiento (formas de almacenar la información) y de operaciones de manipulación de datos dentro de un tipo de entorno informático. Los S.G.B.D. comerciales se basan en un modelo lógico concreto. Ejemplo, Modelo Relacional.
3. **Modelo Físico:** Se utiliza en el diseño físico. Es la implementación física del modelo anterior. Son estructuras de datos a bajo nivel, implementadas dentro de un sistema gestor de base de datos comercial, por ej. Oracle, mysql, etc.,

Los SGBD comerciales se basan en un modelo lógico concreto. Por ej. Oracle en el modelo relacional.

Autoevaluación

¿Cuáles son los modelos que se centran en las operaciones y se implementan en algún sistema gestor de base de datos?

- ☐ Modelo de datos conceptuales.
- ☐ Modelo de datos lógico.
- ☐ Modelo de datos físicos.

No es correcto, estos describen las estructuras de datos y restricciones de integridad.

Efectivamente, un ejemplo sería el modelo relacional.

Incorrecto, estos son estructuras de datos a bajo nivel e implementadas dentro del propio SGBD.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

2.- Terminología del modelo relacional.

Caso práctico



[Ministerio de Educación](#) (Uso educativo)

Ana se pregunta cuál será el modelo con el que se suele trabajar. Actualmente, para la mayoría de las aplicaciones de gestión que utilizan bases de datos, el modelo más empleado es el modelo relacional, por su gran versatilidad, potencia y su base matemática.



[IdITE=195087_im_1.jpg](#) (uso educativo nc)

¿Sabes que el modelo relacional te va a permitir representar la información del mundo real de una manera intuitiva? Así es, pudiendo introducir conceptos cotidianos y fáciles de entender por cualquiera, aunque no sea experto en informática.

El modelo relacional fue propuesto por Edgar Frank Codd en los laboratorios de IBM en California. Como hemos visto, se trata de un modelo lógico que establece una estructura sobre los datos, independientemente del modo en que luego los almacenemos. Es como si guardamos nuestra colección de libros, dependiendo del número de habitaciones que tenga en casa, del tamaño y forma de nuestras estanterías, podremos disponer nuestros libros de un modo u otro para facilitarnos el acceso y consulta. Los libros serán los mismos pero puedo disponerlos de distinta forma.

Es importante recordar que el diseño de las tablas del modelo relacional es la segunda fase del diseño de la base de datos y previamente se ha realizado el diseño conceptual identificando, tras el análisis, las tablas o relaciones resultantes, sus atributos, restricciones y relaciones.

El nombre de modelo relacional viene de la estrecha relación entre el elemento básico de este modelo y el concepto matemático de relación. Si tenemos dos conjuntos A y B, una relación entre estos dos conjuntos sería un subconjunto del producto cartesiano $A \times B$.

El producto cartesiano nos dará la relación de todos los elementos de un conjunto con todos los elementos de los otros conjuntos de ese producto. Al estar trabajando con conjuntos, no puede haber elementos repetidos.

Por ejemplo para los dos conjuntos que se presentan en la imagen, uno denominado Marcas que contiene marcas de coches y otro denominado Modelos que contiene diferentes modelos el resultado de la operación de producto cartesiano será la combinación de todos los elementos de un conjunto con los del otro.

Marcas

Nombre
Nissan
Opel
Toyota

Modelos

Nombre
Qashqai
Corsa
Auris

Producto Cartesiano Marcas X Modelos



Marca.Nombre	Modelo.Nombre
Nissan	Qashqai
Opel	Qashqai
Toyota	Qashqai
Nissan	Corsa
Opel	Corsa
Toyota	Corsa
Nissan	Auris
Opel	Auris
Toyota	Auris

2.1.- Relación o tabla. Tuplas. Dominios.

Pero... ¿qué es eso de “relación”? Hemos dicho que el modelo relacional se basa en el concepto matemático de relación, ya que Codd, que era un experto matemático, utilizó una terminología perteneciente a las matemáticas, en concreto a la teoría de conjuntos y a la lógica de predicados.

Para saber más

Aquí tienes unos enlaces sobre teoría de conjuntos y lógica de predicados:

[Teoría de conjuntos.](#)

[Lógica de predicados.](#)

A partir de ahora, nosotros veremos una relación como una **tabla** con **filas** y **columnas**. Podemos asociar atributos a columnas y tuplas a filas.

✓ **Atributos:** es el nombre de cada dato que se almacena en la relación (tabla). En la tabla Alumnos, que se muestra a la derecha, los atributos o columnas serían DNI, nombre y apellidos..

El nombre del atributo debe describir el significado de la información que representa. En la tabla

Empleados, el atributo **Sueldo** almacenará el valor en euros del sueldo que recibe cada empleado. A veces es necesario añadir una pequeña descripción para aclarar un poco más el contenido. Por ejemplo, si el sueldo es neto o bruto.

✓ **Tuplas:** Se refiere a cada elemento de la relación o tabla. En la tabla alumnos hay 5 tuplas con los atributos DNI, nombre y apellidos de cada uno de los alumnos..

Cada una de las filas de la tabla se corresponde con la idea de registro y tiene que cumplir que:

- ➡ Cada tupla se debe corresponder con un elemento del mundo real.
- ➡ No puede haber dos tuplas iguales (con todos los valores iguales).

Está claro que un atributo en una tupla no puede tomar cualquier valor. No sería lógico que en un atributo Población se guarde "250€". Estaríamos cometiendo un error, para evitar este tipo de situaciones obligaremos a que cada atributo sólo pueda tomar los valores pertenecientes a un conjunto de valores previamente establecidos, es decir, un atributo tiene asociado un **dominio** de valores.

A menudo un dominio se define a través de la declaración de un tipo para el atributo (por ejemplo, diciendo que es un número entero entre 1 y 16), pero también se pueden definir dominios más complejos y precisos. Por ejemplo, para el atributo Sexo de los usuarios, podemos definir un dominio en el que los valores posibles sean "M" o "F" (masculino o femenino).

Tabla: Alumnos

Atributos: →

DNI	NOMBRE	APELLIDOS
1	PEDRO	FERNÁNDEZ
2	LUIS	GÓMEZ
3	PEDRO	GÓMEZ
4	MIGUEL	RIVERO
5	CARLOS	GARCÍA

Tupla →

Jorge Castellanos (Creative Commons CCO)

Una característica fundamental de los dominios es que sean **atómicos**, es decir, que los valores contenidos en los atributos no se pueden separar en valores de dominios más simples.

Un dominio debe tener: **Nombre, Definición lógica, Tipo de datos y Formato.**

Por ejemplo, si consideramos el Sueldo de un empleado, tendremos:

- ✔ **Nombre:** Sueldo.
- ✔ **Definición lógica:** Sueldo neto del empleado
- ✔ **Tipo de datos:** número entero.
- ✔ **Formato:** 9.999€.

Autoevaluación

¿Cuáles de las siguientes afirmaciones son ciertas sobre las tuplas y los atributos?

- ☐ Las tuplas deben corresponderse con un elemento del mundo real.

- ☐ Podríamos tener dos o más tuplas iguales.

- ☐ Un atributo se define en un dominio de valores.

- ☐ El nombre de cada dato que se almacena en la relación se denomina Atributo.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto

2.2.- Grado. Cardinalidad.

Ya hemos visto que una relación es una tabla con filas y columnas. Pero ¿hasta cuántas columnas puede contener? ¿Cuántos atributos podemos guardar en una tabla?

Llamaremos **grado** al tamaño de una tabla en base a su número de atributos (columnas). Mientras mayor sea el grado, mayor será la complejidad para trabajar con ella.

¿Y cuántas tuplas (filas o registros) puede tener?

Llamaremos **cardinalidad** al número de tuplas o filas de una relación o tabla.

Vamos a verlo con un ejemplo. Dadas las relaciones

$A=\{\text{Carlos, María}\}$, $B=\{\text{Matemáticas, Lengua}\}$, $C=\{\text{Aprobado, Suspenso}\}$.

Las posibles relaciones que obtenemos al realizar el producto cartesiano $A \times B \times C$ da como resultado una relación de grado 3, ya que tiene 3 columnas

Producto Cartesiano $A \times B \times C$.

$A=\{\text{Carlos, María}\}$	$B=\{\text{Matemáticas, Lengua}\}$	$C=\{\text{Aprobado, Suspenso}\}$
CARLOS	MATEMÁTICAS	APROBADO
CARLOS	MATEMÁTICAS	SUSPENSO
CARLOS	LENGUA	APROBADO
CARLOS	LENGUA	SUSPENSO
CARLOS	INGLÉS	APROBADO
CARLOS	INGLÉS	SUSPENSO
MARÍA	MATEMÁTICAS	APROBADO
MARÍA	MATEMÁTICAS	SUSPENSO
MARÍA	LENGUA	APROBADO
MARÍA	LENGUA	SUSPENSO
MARÍA	INGLÉS	APROBADO
MARÍA	INGLÉS	SUSPENSO

Si cogemos un subconjunto de ésta con 5 filas, tendríamos una relación de cardinalidad 5. Por ej.

**Subconjunto del Producto Cartesiano $A \times B \times C$ con
cardinalidad 5.**

A={Carlos, María}	B={Matemáticas, Lengua}	C={Aprobado, Suspenso}
CARLOS	MATEMÁTICAS	APROBADO
CARLOS	LENGUA	APROBADO
CARLOS	INGLÉS	APROBADO
MARÍA	MATEMÁTICAS	APROBADO
MARÍA	INGLÉS	SUSPENSO

2.3.- Sinónimos.

Caso práctico



[Ministerio de Educación](#) (Uso educativo)

Ana está un poco liada con tantos términos nuevos. ¿Si Juan habla de tuplas se está refiriendo a registros? Los registros eran las filas de las las tablas, ¿no? Será mejor que hagamos un resumen.

Los términos vistos hasta ahora tienen distintos sinónimos según la nomenclatura utilizada. Trabajaremos con tres:

- ✓ **En el modelo relacional:** RELACIÓN - TUPLA - ATRIBUTO - GRADO - CARDINALIDAD.
- ✓ **En tablas:** TABLA - FILA - COLUMNAS - NÚMERO COLUMNAS - NÚMERO FILAS.
- ✓ **En términos de registros:** FICHEROS - REGISTROS - CAMPOS - NÚMERO CAMPOS - NÚMERO REGISTROS.



[IdITE=23712_10_m_1.jpg](#) (Uso educativo nc)

Nomenclatura relacional		Nomenclatura tablas		Nomenclatura ficheros
relación	=	tabla	=	fichero
tupla	=	fila	=	registro
atributo	=	columna	=	campo
grado	=	nº columnas	=	nº campos
cardinalidad	=	nº filas	=	nº registros

[Ministerio de Educación y FP](#) (Uso educativo nc)

Autoevaluación

Relaciona cada término del modelo relacional con la terminología de Tablas.

Sinónimos.

Terminología del modelo relacional.	Relación.	Terminología en Tablas.
RELACIÓN	☐	1. COLUMNAS
TUPLA	☐	2. NÚMERO DE COLUMNAS
ATRIBUTO	☐	3. NÚMERO DE FILAS
GRADO	☐	4. FILA
CARDINALIDAD	☐	5. TABLA

Enviar

Debes tener en cuenta que a partir de ahora emplearemos frecuentemente estos términos.

3.- Relaciones. Características de una relación (tabla).

Caso práctico



[Ministerio de educación](#) (Uso educativo nc)

Juan, tras su análisis y varios días de trabajo, ha obtenido las relaciones con las que trabajará y los atributos que desea guardar en la base de datos. Junto con Ana va a repasar que se cumplan todas las propiedades y así asegurarse que el modelo es el adecuado. También necesitará saber qué información podrá consultar el usuario para así crear algunas tablas de modo temporal.

¿En un modelo relacional se puede utilizar cualquier relación? ¿Es válida cualquier tabla o se deben cumplir algunas propiedades?

Debes saber que:

- ✓ Cada tabla tiene un **nombre distinto**.



[Ministerio de Educación](#) (Uso educativo nc)

- ✓ Como hemos visto antes, cada atributo (columna) de la tabla **toma un solo valor** en cada tupla (fila).
- ✓ Cada atributo (columna) tiene un **nombre distinto** en cada tabla (pero puede ser el mismo en tablas distintas).
- ✓ **No** puede haber dos tuplas (filas) **completamente iguales**.

Carlos	Matemáticas	Aprobado
Carlos	Matemáticas	Suspense
Carlos	Lengua	Aprobado
Carlos	Lengua	Aprobado

Pilar Ramírez. (Uso educativo nc)

- ✓ El **orden** de las tuplas (filas) **no importa**.

a	20
b	30
c	70

=

a	20
c	70
b	30

Pilar Ramírez. (Uso educativo nc)

- ✓ El **orden** de los atributos (columnas) **no importa**.

a	20
b	30
c	70

=

20	a
70	c
30	b

Pilar Ramírez. (Uso educativo nc)

- ✓ Todos los datos de un atributo (columna) deben ser del **mismo dominio**. Si hemos definido que el dominio del atributo Nota sólo admite los valores Aprobado o Suspenso entonces el dato NOTABLE sería incorrecto ya que no pertenece al dominio.

Carlos	Matemáticas	Aprobado
Carlos	Lengua	Suspenso
Carlos	Inglés	NOTABLE
María	Matemáticas	Suspenso
María	Lengua	Suspenso

Pilar Ramírez. (Uso educativo nc)

Autoevaluación

¿Cuál de las siguientes afirmaciones no es cierta en una relación?

- ☐ Todos los atributos deben estar en el mismo dominio.
- ☐ No puede haber dos tuplas completamente iguales.
- ☐ Cada atributo de la tabla toma un único valor en cada tupla.
- ☐ Podemos tener tablas con el mismo nombre en la misma base de datos.

No es correcto. Los valores de los atributos deben pertenecer al dominio que tienen definido.

No es la respuesta correcta, si hubiese dos tuplas iguales estaríamos infringiendo el modelo relacional.

Incorrecto, debe ser así para todas las tuplas.

Correcto, sí es posible en bases de datos distintas.

Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

3.1.- Tipos de relaciones (tablas).

Caso práctico



[Ministerio de Educación](#) (Uso educativo no)

Juan le está contando a Ana que hay que distinguir las relaciones en función del uso que se le vaya a dar. Tal y como han hablado con el cliente, sabe que unos jugadores accederán a un tipo de tablas como usuarios, y las personas que administran la base de datos lo harán a otras. Es obvio que tenemos que distinguir entre unas y otras.

Existen varios tipos de relaciones o tablas y las vamos a clasificar en:

- ✔ **Persistentes:** Sólo pueden ser borradas por los usuarios.
 - ➡ **Base:** Independientes, se crean indicando su estructura y sus ejemplares (conjunto de tuplas o filas).
 - ➡ **Vistas:** son tablas que sólo almacenan una definición de consulta, resultado de la cual se obtiene datos que proceden de otras tablas base o de otras vistas e instantáneas. Si los datos de las tablas base cambian, los de la vista que utilizan esos datos también cambiarán ya que se obtienen a partir de ellas.
 - ➡ **Instantáneas:** son vistas (se crean de la misma forma) pero sí almacenan los datos que muestran, además de la consulta que la creó. Solo modifican su resultado cuando el sistema se refresca cada cierto tiempo. Es como una fotografía de la relación, que sólo es válida durante un periodo de tiempo concreto.
- ✔ **Temporales:** Son tablas que son eliminadas automáticamente por el sistema.

Autoevaluación

Las relaciones que se crean indicando su estructura y sus ejemplares se denominan:

- ☐ Instantáneas.
- ☐ Vistas.
- ☐ Base.

No es correcto, las instantáneas son un tipo de vistas.

Incorrecto, las vistas solo almacenan la definición de la consulta.

Correcto, sigue así.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

4.- Tipos de datos.

Caso práctico

Juan le ha pedido a Ana que repase cada una de las relaciones y en función de los datos que contenga cada atributo, elija el tipo de datos más adecuado. Más adelante habrá que restringir esos valores para que al introducir datos no se produzcan errores. Los tipos de datos ocupan espacio en el disco duro delservidor donde se guarde y pueden hacer que el acceso sea más lento, así que hay que optimizar.



[Ministerio de Educación](#) (Uso educativo no)

Además, Ana todavía recuerda aquella vez que tuvo que entregar una práctica en la facultad sobre base de datos. Guardó el teléfono con un formato de número y cuando fue a imprimir un informe... ¡no quiere ni acordarse! Le salieron unos números de teléfonos que nada tenían que ver con los datos introducidos.

¿Qué es un DNI? ¿Con qué datos lo representamos? El DNI es una información que es susceptible de ser guardada. Normalmente el DNI está formado por dígitos y una letra al final. Si tuviéramos que clasificarlo diríamos que es un conjunto de caracteres alfanuméricos. ¿Y si pensamos en Sueldo? Aquí lo tenemos un poco más claro, evidentemente es un número entero o con decimales.

Hasta ahora hemos visto que vamos a guardar información relacionada en forma de filas y columnas. Las columnas son los atributos o información que nos interesa incluir del mundo real que estamos modelando.

Hemos visto que esos atributos se mueven dentro de un dominio, que formalmente es un conjunto de valores. Pues bien, en términos de sistemas de base de datos, se especifica indicando el tipo de dato (de forma general) y el conjunto de valores que puede tomar (de forma restringida). El conjunto de valores restringidos se le indicará en la definición de la tabla si el SGBD lo permite.

Por ejemplo para un atributo que va a guardar el género (M masculino o F femenino), el tipo de datos será carácter o texto de longitud 1. Los valores posibles M o F se indicarán en la definición de la tabla.

Al crear la relación (tabla) decidimos qué conjunto de datos deberá ser almacenado en los atributos de las filas. Tenemos que asignar un tipo de dato a cada atributo.

Con la asignación de tipos de datos, también habremos seleccionado un dominio para cada atributo.

Cada campo:

- ✓ debe poseer un **Nombre** (relacionado con los datos que va a contener) y
- ✓ debe tener asociado un **Tipo de dato** que determinará qué valores puede tomar y qué operaciones se pueden realizar con ellos.

Existen distintas formas de nombrar los tipos de datos dependiendo del lenguaje que utilicemos (C, Java, PHP, MySQL, SQL, Pascal, etc.).

Veamos cuales son los tipos de datos más comunes y habituales, existentes en la mayoría de los lenguajes, que posteriormente se definirán utilizando la sintaxis específica del lenguaje elegido.

- ✓ **Texto:** almacena cadenas (conjunto) de caracteres: números con los que **no** vamos a realizar operaciones matemáticas, letras o símbolos.
- ✓ **Numérico:** almacena números. Si dudamos entre numérico o texto tendremos en cuenta si vamos a realizar operaciones matemáticas con ellos, en cuyo caso será numérico.
- ✓ **Fecha/hora:** almacena fechas y horas.
- ✓ **Sí/No:** almacena datos que solo tienen dos posibilidades (verdadero/falso).
- ✓ **Autonumérico:** se puede considerar un subtipo de Numérico ya que almacena valores numéricos secuenciales que el SGBD incrementa de modo automático al añadir un registro (fila).
- ✓ **Memo:** almacena texto largo (mayor que un tipo texto).
- ✓ **Moneda:** pero con una característica especial, y es que los valores representan cantidades de dinero.
- ✓ **Objeto OLE:** almacena gráficos, imágenes o textos creados por otras aplicaciones.

Para determinar qué tipo de dato se asocia a cada atributo se tiene en cuenta el conjunto de valores que puede tomar y las operaciones que hay que realizar con él.

Un código postal, por ejemplo 06800, a pesar de estar formado por dígitos numéricos, es mejor definirlo como una cadena de 5 caracteres por dos motivos: porque no se va a realizar operaciones matemáticas con él y porque los ceros a la izquierda no deben ser obviados como si fuera numérico. El número de teléfono se encuentra en un caso similar.

Es fundamental determinar de forma correcta el tipo de dato y tamaño para cada atributo, ya que si se define mal, no permitirá almacenar la información deseada o puede que almacene información incompleta. Por ejemplo, si se define de un tamaño o longitud inferior al valor que va a contener. Supongamos que nos precipitamos y definimos el DNI como un campo de tipo cadena de 8 caracteres; si se quiere registrar el DNI 89234432B que tiene 9 caracteres, es posible que el sistema lo almacene mal, dejando atrás el carácter sobrante. De ahí la importancia de este proceso.

Para saber más

Si quieres saber un poco más sobre los tipos de datos generales puedes ver este enlace de Wikipedia:

[Tipo de datos.](#)

En el siguiente enlace puedes ver cómo se definen los tipos de datos en el SGBD Mysql

[Tipos de datos en el SGBD MySQL](#)

5.- Claves.

Caso práctico



[Ministerio de Educación](#) (Uso educativo nc)

Juan está revisando la relación Usuarios. En esta tabla va a guardar los siguientes atributos: Login del jugador que será nuestro usuario, Password o Contraseña, Nombre y Apellidos, Dirección, Código Postal, Localidad, Provincia, País, Fecha de nacimiento para comprobar que no es menor de edad, Fecha de ingreso en la web, Correo electrónico, Sexo y por último los Créditos (dinero "ficticio") que tenga.

¿Cómo diferenciamos unos usuarios de otros? ¿Cómo sabemos que no estamos recogiendo la misma información? ¿Cómo vamos a distinguir unas tuplas de otras? Lo haremos mediante los valores de sus atributos. Para ello, buscaremos un atributo o un conjunto de atributos que identifiquen de modo único las tuplas (filas) de una relación (tabla). A ese atributo o conjunto de atributos lo llamaremos **superclaves**.



[IdITE=109884](#) (Uso educativo nc)

Hemos visto que una característica de las tablas era que no puede haber dos tuplas (filas) completamente iguales, con lo que podemos decir que toda la fila como conjunto sería una superclave.

Por ejemplo, en la tabla Usuarios tenemos las siguientes superclaves:

- ✓ {Nombre, Apellidos, login, e_mail, F_nacimiento}
- ✓ {Nombre, Apellidos, login, e_mail}
- ✓ {login, e_mail}
- ✓ {login}

Tendríamos que elegir alguna de las superclaves para diferenciar las tuplas. En el modelo relacional trabajamos con tres tipos de claves:

- ✓ Claves **candidatas**.
- ✓ Claves **primarias**.
- ✓ Claves **alternativas**.
- ✓ Claves **ajenas**.

A continuación veremos cada una de ellas.

5.1.- Clave candidata. Clave primaria. Clave alternativa.

Si puedo elegir entre tantas claves, ¿con cuál me quedo? Tendremos que elegir entre las claves "candidatas" la que mejor se adapte a mis necesidades. ¿Y cuáles son éstas? Las claves **candidatas** serán aquel conjunto de atributos que identifiquen de manera única cada tupla (fila) de la relación (tabla). Es decir, las columnas cuyos valores no se repiten en ninguna otra fila de la tabla. Por tanto, cada tabla debe tener **al menos una clave candidata** aunque puede haber más de una.



[Anonymous](#) (CC BY)

Siguiendo con nuestro ejemplo, podríamos considerar los atributos Login o E_mail como claves candidatas, ya que sabemos que el Login debe ser único para cada usuario, a E_mail le sucede lo mismo. Pero también cabe la posibilidad de tomar: Nombre, Apellidos y F_nacimiento, las tres juntas como clave candidata.

Las claves candidatas pueden estar formadas por más de un atributo, siempre y cuando éstos identifiquen de forma única a la fila. Cuando una clave candidata está formada por más de un atributo, se dice que es una **clave compuesta**.

Una clave **candidata** debe cumplir los siguientes requisitos:

- ✔ **Unicidad:** no puede haber dos tuplas (filas) con los mismos valores para esos atributos.
- ✔ **Irreductibilidad:** si se elimina alguno de los atributos deja de ser única.

Si elegimos como clave candidata **Nombre, Apellidos y F_nacimiento**, cumple con la unicidad puesto que es muy difícil encontrarnos con dos personas que tengan el mismo nombre, apellidos y fecha de nacimiento iguales. Es irreductible puesto que sería posible encontrar dos personas con el mismo nombre y apellidos o con el mismo nombre y fecha de nacimiento, por lo que son necesarios los tres atributos (campos) para formar la clave.

Para identificar las claves candidatas de una relación no nos fijaremos en un momento concreto en el que vemos una base de datos. Puede ocurrir que en ese momento no haya duplicados para un atributo o conjunto de atributos, pero esto no garantiza que se puedan producir. El único modo de identificar las claves candidatas es conociendo el significado real de los atributos (campos), ya que así podremos saber si es posible que aparezcan duplicados. Es posible desechar claves como candidatas fijándonos en los posibles valores que podemos llegar a tener. Por ejemplo, podríamos pensar que Nombre y Apellidos podrían ser una clave candidata, pero ya sabemos que cabe la posibilidad de que dos personas puedan tener el mismo Nombre y Apellidos, así que lo descartamos.

Hasta ahora, seguimos teniendo varias claves con la que identificamos de modo único nuestra relación. De ahí el nombre de candidatas. Hemos de quedarnos con una.

La **clave primaria** de un relación es aquella clave candidata que se escoge para identificar sus tuplas de modo único. Ya que una relación no tiene tuplas duplicadas, siempre hay una clave candidata y, por lo tanto, la relación siempre tiene clave primaria. En el peor caso, la clave primaria estará formada por todos los atributos de la relación, pero normalmente habrá un pequeño subconjunto de los atributos que haga

esta función. En otros casos, podemos crear un campo único que identifique las tuplas, por ejemplo un código de usuario, que podrían estar constituidos por valores autonuméricos.

Las claves candidatas que no son escogidas como clave primaria son denominadas **claves alternativas**.

Si en nuestra tabla Usuarios escogemos Login como clave primaria, el E_mail o {Nombre, Apellidos, F_Nacimiento} serán nuestras claves alternativas.

Autoevaluación

Rellena los huecos con los conceptos adecuados.

Dentro del conjunto de superclaves, se llaman claves a aquellas que identifican unívocamente a cada una de las . De entre éstas, escogeremos la clave . Aquellas que no escogemos se denominarán claves .

Enviar

5.2.- Clave externa, ajena o secundaria.



[Nathan Eady / liftarn](#) (Open Clip Art .CC)

Hasta ahora no nos hemos planteado cómo se relacionan unas tablas con otras dentro de una base de datos. Si tenemos las tablas Usuarios y Partidas, necesariamente habrá una "relación" entre ellas. Deben compartir algún dato en común que las relacione. Una partida es jugada por un jugador (Usuarios), por lo que en la tabla Partida deberíamos guardar algún dato del usuario-jugador, pero ¿cuál?

Una clave **ajena**, también llamada **externa**, **foránea** o **secundaria**, es un atributo o conjunto de atributos de una relación cuyos valores coinciden con los valores de la clave primaria de alguna otra relación (o de la misma). Las claves ajenas **representan relaciones entre datos**. Dicho de otra manera, son los datos de atributos de una tabla cuyos valores están relacionados con atributos de otra tabla.

En la tabla Partidas, se recogen datos como **Cod_partida**, **Fecha** y **Hora de creación**, **Nombre de la partida**, etc. ¿Qué campo utilizaremos para relacionarla con la tabla Usuarios? Si nos basamos en la definición, deberíamos utilizar la clave primaria de la tabla Usuarios. Por tanto, el atributo Login que es la clave principal en su tabla aparecerá en la tabla Partidas como clave ajena, externa o secundaria. El Login en Partidas hace referencia a cada jugador que juega esa partida. En lugar de guardar todos los datos de ese jugador en la misma tabla, lo hacemos en otra y lo "referenciamos" por su clave primaria tomándola como ajena.

Es lógico que las claves ajenas no tengan las mismas propiedades y restricciones que tienen como clave primaria en su tabla, por tanto, sí que pueden repetirse en la tabla. En nuestro ejemplo, un mismo jugador puede jugar varias partidas.

Las claves ajenas tienen por objetivo establecer una conexión con la clave primaria que referencian. Por lo tanto, **los valores de una clave ajena deben estar presentes como clave primaria en la tabla a la que hacen referencia , o bien deben ser valores nulos**. En caso contrario, la clave ajena representaría una referencia o conexión incorrecta, lo que supondría que la información almacenada es inconsistente (no fiable). Imagina un código de jugador en la tabla Partidas que no se corresponde con ningún jugador de la tabla Usuarios.

No podemos tener una partida de un jugador que previamente no se ha registrado y no existe en la tabla Usuarios. Pero sí podemos tener los datos de una partida y desconocer el jugador de ésta, es decir la columna jugador puede tener el valor nulo (sin valor).

Autoevaluación

¿Cuáles de las siguientes afirmaciones sobre las claves ajenas son correctas?

- ☐ Puede "referenciar" a la clave primaria de la misma tabla donde se encuentra.

☐ Puede "referenciar" a la clave primaria de otra tabla.

☐ Representa relaciones entre datos.

☐ Puede contener valores nulos.

☐ No puede repetirse en la tabla.

Mostrar retroalimentación

Solución

1. Correcto
2. Correcto
3. Correcto
4. Correcto
5. Incorrecto

Para saber más

Interesante artículo sobre las claves ajenas y su importancia:

[De las claves ajenas, foráneas, externas...](#)

Si necesitas refrescar o simplemente aprender el concepto de clave primaria, en la wikipedia puedes consultarlo:

[Claves primarias.](#)

6.- Índices. Características.

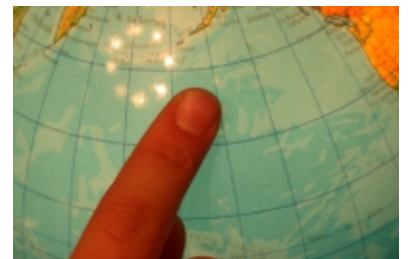
Caso práctico



[Ministerio de Educación](#) (Uso educativo)

Juan considera que es beneficioso crear un índice para la tabla Usuarios. Podría agilizar las búsquedas de usuarios registrados. Le ha contado a Ana que es conveniente tenerlo, aunque también le ha explicado que tener muchos índices no es bueno. Tendrán que hacer una buena elección del número de índices que van a manejar.

Imagina que estás creando un diccionario de términos informáticos. Podrías elegir la opción de escribirlo en una única hoja muy larga (estilo pergamino) o bien distribuirlo por hojas. Está claro que lo mejor sería distribuirlo por páginas. Y si buscamos el término "informática" en nuestro diccionario, podríamos comenzar a buscar en la primera página y continuar una por una hasta llegar a la palabra correspondiente. O bien crear un índice al principio, de manera que podamos consultar a partir de qué página podemos localizar las palabras que comienzan por "i". Esta última opción parece la más lógica.



[Id!TE=111373](#) (Uso educativo no)

Pues bien, en las bases de datos, cada tabla se divide internamente en páginas de datos, y se define el índice a través de un campo (o campos) y es a partir de este campo desde donde se busca.

Un **índice** es una estructura de datos que permite acceder a diferentes filas de una misma tabla a través de un campo o campos . Esto permite un acceso mucho más rápido a los datos.

Los índices son útiles cuando se realizan consultas frecuentes a un rango de filas o una fila de una tabla. Por ejemplo, si consultamos los usuarios cuya fecha de ingreso es anterior a una fecha concreta.

Los cambios en los datos de las tablas (agregar, actualizar o borrar filas) son incorporados automáticamente a los índices con transparencia total.

Debes saber que los índices son independientes, lógica y físicamente de los datos, es por eso que pueden ser creados y eliminados en cualquier momento, sin afectar a las tablas ni a otros índices.

¿Cuándo indexamos? No hay un límite de columnas a indexar, si quisiéramos podríamos crear un índice para cada columna, pero no sería operativo. Normalmente tiene sentido crear índices para ciertas columnas ya que agilizan las operaciones de búsqueda de base de datos grandes. Por ejemplo, si la información de nuestra tabla Usuarios se desea consultar por apellidos a menudo y se necesita agilidad en

los accesos, tiene sentido indexar por esa columna. No conviene indexar por columnas de gran tamaño porque puede resultar contraproducente.

Al crear índices, las operaciones de modificar o agregar datos se ralentizan, ya que al realizarlas es necesario actualizar tanto la tabla como el índice. Por tanto hay que pensar bien cuándo interesa definir o no un índice.

Si se elimina un índice, el acceso a datos puede ser más lento a partir de ese momento.

El SGBD utiliza índices para la gestión de las claves ajenas y de las claves primarias. En el caso de las claves primarias serán índices únicos (no admiten valores repetidos).

Para saber más

Si quieres conocer más sobre los índices y Mysql puedes leer este artículo:

[Índices](#)

7.- El valor NULL. Operaciones con este valor.

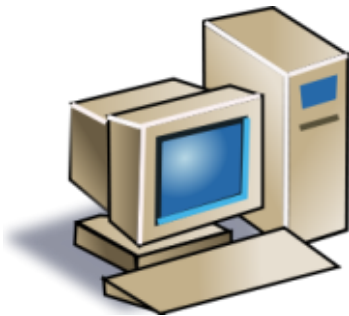
Caso práctico

Ana tiene un poco más claro el concepto de relación y las características de los atributos. Sabe que éstos se definen en un dominio. Pero ¿qué ocurre si no conozco algún valor de un dato? ¿Si en la tabla de usuarios estoy pidiendo que se guarde el sexo y el jugador no quiere decirlo? ¿Qué puede ocurrir? Si se permite que ese dato no sea obligatorio lo que me quedaría sería un dato vacío de información.



[Ministerio de Educación](#) (Uso educativo no)

Vamos a ver que eso es posible y que ese valor tiene una denominación propia.



[rgtaylor_csc](#) (Open Clip Art .CC)

¿Qué sucede si al guardar los datos de los Usuarios hay algún dato que no tengo o no necesito guardarlo porque no corresponde?

Independientemente del dominio al que pertenezca un campo, éste puede tomar un valor especial denominado **NULO** (**NULL** en inglés) que designará la ausencia de dato.

Cuando por cualquier motivo se desconoce el valor de un campo, por ejemplo, desconocemos el teléfono del usuario, o bien ese campo carece de sentido (siguiendo con el mismo ejemplo, puede que el usuario no tenga teléfono), podemos asignar a ese campo el valor especial NULO.

Cuando trabajamos con claves secundarias el valor nulo indica que la tupla o fila no está relacionada con ninguna otra tupla o fila. Este valor NULO es común a cualquier dominio.

Pero ten en cuenta una cosa, no es lo mismo valor NULO que ESPACIO EN BLANCO.

Tampoco será lo mismo valor NULO que el valor CERO.

Un ordenador tomará un espacio en blanco como un carácter como otro cualquiera. Por tanto, si introducimos el carácter "espacio en blanco" estaríamos introduciendo un valor que pertenecería al dominio **texto** y sería distinto al concepto "ausencia de valor" que sería **no incluir nada** (nulo).

Este valor se va a utilizar con frecuencia en las bases de datos y es imprescindible saber cómo actúa cuando se emplean operaciones lógicas sobre ese valor. En la lógica booleana tenemos los valores VERDADERO y FALSO, pero un valor NULO no es ni verdadero ni falso.

Cuando necesitemos comparar dos campos, si ambos son nulos no podremos obtener ni verdadero ni falso. Necesitaremos definir la lógica con este valor. Veamos los operadores lógicos más comunes y sus resultados utilizando el valor nulo:

- ✓ VERDADERO Y (AND) NULO daría como resultado NULO.
- ✓ FALSO Y (AND) NULO daría como resultado FALSO.
- ✓ VERDADERO O (OR) NULO daría como resultado VERDADERO.
- ✓ FALSO O NULO daría como resultado NULO.
- ✓ NO (NOT) NULO daría como resultado NULO.

En todas las bases de datos relacionales se utiliza un operador llamado IS NULL (ES NULO) que devuelve VERDADERO si el valor con el que se compara es NULO.

Para saber más

El uso del valor nulo es un tema que da mucho que hablar, aquí puedes leer sobre ello:

[Valores nulos](#)

Autoevaluación

¿Cuáles de las siguientes afirmaciones sobre el valor nulo son ciertas?

☐ Designa ausencia de dato.

☐ Es lo mismo que espacio en blanco.

☐ Es lo mismo que cero.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Incorrecto

8.- Vistas.

Caso práctico



[Ministerio de Educación](#) (Uso educativo nc)

Ana lleva un buen rato pensando cómo hacer si necesitara consultar datos de dos tablas distintas, por ejemplo, sería interesante obtener los nombres de los usuarios que estén jugando una determinada partida. O quizás consultar otros datos por el estilo. ¿Cómo lo hace si ya están definidas las tablas del modelo? ¿Cómo crear esas tablas? Juan le va a explicar que esa información la puede obtener a través de las vistas.

Cuando vimos los distintos tipos de relaciones, aprendimos que, entre otros, estaban las vistas. Ahora ya tenemos más conocimientos para comprender mejor este concepto.

Una **vista** es una tabla "virtual" cuyas filas y columnas se obtienen a partir de una o de varias tablas que constituyen nuestro modelo. Lo que se almacena no es la tabla en sí, sino su definición, por eso decimos que es "virtual". Una vista actúa como filtro de las tablas a las que hace referencia en ella.

La consulta que define la vista puede provenir de una o de varias tablas, o bien de otras vistas de la base de datos actual u otras bases de datos.

No existe ninguna restricción a la hora de consultar vistas pero sí hay algunas restricciones a la hora de modificar los datos de éstas establecidas para mantener la integridad y consistencia de los datos.

Podemos dar dos razones por las que queramos crear vistas:

- ✓ **Seguridad**, nos puede interesar que los usuarios tengan acceso a una parte de la información que hay en una tabla, pero no a toda la tabla.
- ✓ **Comodidad**, como veremos al pasar nuestras tablas/relaciones a un lenguaje de base de datos, puede que tengamos que escribir sentencias bastante complejas, las vistas no son tan complejas.

Las vistas no tienen una copia física de los datos, son sentencias de consultas a los datos que hay en las tablas, por lo que si actualizamos los datos de una vista, estamos actualizando realmente la tabla, y si actualizamos la tabla estos cambios serán visibles desde la vista.

Aunque **no siempre** podremos actualizar los datos de una vista, dependerá de la complejidad de la misma y del gestor de base de datos. No todos los gestores de bases de datos permiten actualizar vistas, Oracle, por ejemplo, no lo permite, mientras que SQL Server sí.



[IdITE=111266](#) (Uso educativo nc)

Autoevaluación

Una vista puede proceder de:

☐ Una tabla.

☐ Varias tablas.

☐ Otras vistas de la misma base de datos.

☐ Otras vistas de otras bases de datos.

Mostrar retroalimentación

Solución

1. Correcto
2. Correcto
3. Correcto
4. Correcto

9.- Usuarios. Roles. Privilegios

Caso práctico



[Ministerio de Educación](#) (Uso educativo nc)

Juan debe consultar al cliente qué usuarios van a acceder a la base de datos y qué privilegios se les va a otorgar. Esta parte es primordial si queremos salvaguardar el contenido de la base de datos. ¿Qué ocurriría si cualquiera pudiera ver la información personal de todos los usuarios registrados? Estaríamos cometiendo un fallo de seguridad además de incumplir la ley de protección de datos.

A la hora de conectarnos a la base de datos es necesario que utilicemos un modo de acceso, de manera que queden descritos los permisos de que dispondremos durante nuestra conexión. En función del nombre de usuario tendremos unos permisos u otros.

Un **usuario** es un conjunto de permisos que se aplican a una conexión de base de datos.

Tiene además otras características:

- ✓ Es el propietario de ciertos objetos (tablas, vistas, etc.).
- ✓ Realiza las copias de seguridad.
- ✓ Tiene asignada una cuota de almacenamiento.
- ✓ Tiene asignado untablespace por defecto para los objetos en Oracle.

Pero no todos los usuarios deberían poder hacer lo mismo cuando acceden a la base de datos. Por ejemplo, unadministrador debería tener más privilegios que un usuario que quiere realizar una simple consulta.

¿Qué es un **privilegio**? No es más que un permiso dado a un usuario para que realice ciertas operaciones, que pueden ser de dos tipos:

- ✓ **De sistema:** necesitará el permiso de sistema correspondiente.
- ✓ **Sobre objeto:** necesitará el permiso sobre el objeto en cuestión.

¿Y no sería interesante poder agrupar esos permisos para darlos juntos? Para eso tenemos el rol.

Un **rol** de base de datos no es más que una agrupación de permisos de sistema y de objeto.

Podemos tener a un grupo determinado de usuarios que tengan permiso para consultar los datos de una tabla concreta y no tener permiso para actualizarlos. Luego un rol permite asignar un grupo de permisos a



[Francisco Mochis](#) (Dominio público)

un usuario. De este modo, si asignamos un rol con 5 permisos a 200 usuarios y luego queremos añadir un permiso nuevo al rol, no tendremos que ir añadiendo este nuevo permiso a los 200 usuarios, ya que el rol se encarga de propagarlo automáticamente.

Autoevaluación

Rellena los huecos con los conceptos adecuados.

Al nombre utilizado para acceder a una base de datos, se le llama . Los permisos dados a usuarios para que realicen ciertas operaciones se les llama . Si tengo una agrupación de permisos juntos, tenemos un .

Enviar

Caso práctico



[Ministerio de Educación](#) (Uso educativo no)

Hasta ahora Ana y Juan no han tenido que utilizar mucho el ordenador, ya es hora de ponerse manos a la obra. El diseño está casi finalizado y ahora es necesario pasarlo a un lenguaje adecuado. Juan



[Ministerio de Educación](#) (Uso educativo no)

había acordado con Ada que usarían Oracle como SGBD. Para trabajar con esta aplicación es necesario tener conocimientos del lenguaje que utiliza, en concreto SQL para Oracle, que tiene ciertas variaciones con el estándar. Ana está deseando comenzar a introducir los datos necesarios.

SQL (Structured Query Language) es un lenguaje de dominio específico utilizado en programación, diseñado para administrar, y recuperar información de sistemas de gestión de bases de datos relacionales. Es el lenguaje fundamental de los SGBD relacionales. Es uno de los lenguajes más utilizados en informática en todos los tiempos. Es un lenguaje declarativo y por tanto, lo más importante es definir **qué** se desea hacer, y **no cómo** hacerlo. De esto último ya se encarga el SGBD.

Hablamos por tanto de un lenguaje normalizado que nos permite trabajar con cualquier tipo de lenguaje (ASP o PHP) en combinación con cualquier tipo de base de datos (Access, SQL Server, MySQL, MariaDB, Oracle, etc.).

El hecho de que sea estándar no quiere decir que sea idéntico para cada base de datos. Así es, determinadas bases de datos implementan funciones específicas que no tienen necesariamente que funcionar en otras.

Aunque SQL está estandarizado, siempre es recomendable revisar la documentación del SGBD con el que estemos trabajando para conocer su sintaxis concreta, ya que algún comando, tipo de dato, etc., puede no seguir el estándar.

SQL posee dos características muy apreciadas, **potencia** y **versatilidad**, que contrastan con su facilidad para el aprendizaje, ya que utiliza un lenguaje bastante natural. Es por esto que las instrucciones son muy parecidas a órdenes humanas. Por esta característica se le considera un Lenguaje de Cuarta Generación.

Aunque frecuentemente oigas que SQL es un "lenguaje de consulta", ten en cuenta que no es exactamente solo eso ya que contiene muchas otras capacidades además de la de consultar la base de

datos:

- ✔ la definición de la propia estructura de los datos (DDL)
- ✔ su manipulación (DML)
- ✔ y la especificación de conexiones seguras (DCL)

Por tanto, el lenguaje estructurado de consultas SQL es un lenguaje que permite operar con los datos almacenados en las bases de datos relacionales.

Se puede trabajar de dos formas con SQL:

- ✔ SQL embebido: las sentencias se escriben dentro de un programa escrito en otro lenguaje como Java, PHP, etc..
- ✔ SQL interpretado: Podemos usar un entorno gráfico para escribir y ejecutar las sentencias (nosotros utilizaremos WorkBench) o bien desde [MySql terminal client](#) que es el programa de línea de comandos de Oracle que permite ejecutar comandos SQL y PL/SQL de forma interactiva.

En la unidad 1 utilizaste SQLPlus para conectarte a la base de datos y crear tu usuario. Es importante que tengas soltura en su uso, aunque manejes de forma más habitual el interface gráfico WorkBench, ya que permite hacer más operaciones.

En el Anexo III de esta unidad y en el siguiente enlace tienes los pasos a seguir para descargarte e instalarte WorkBench y dar los primeros pasos:

<https://www.youtube.com/embed/KvC7iM6O4ik>

[WorkBench: Instalación y primeros pasos](#) (pdf - 964 KB)

Para saber más

Ya hemos llegado a los lenguajes de quinta generación, en el siguiente enlace puedes ver qué caracteriza a los lenguajes de cada generación :

[Generaciones de lenguajes de programación](#)

En este enlace encontrarás de una manera breve, pero interesante, la historia del SQL.

[SQL.](#)

Son muchas las razones por las que es importante aprender SQL y tener destreza en su uso.

En el siguiente enlace tienes 7

[Siete buenas razones para aprender SQL](#)

10.1.- Elementos del lenguaje. Normas de escritura.

Imagínate que cada programador utilizara sus propias reglas para escribir. Esto sería un caos. Es muy importante establecer los elementos con los que vamos a trabajar y unas normas que seguir.

El lenguaje SQL está compuesto por comandos, cláusulas, operadores, funciones y literales. Todos estos elementos se combinan en las instrucciones o sentencias y se utilizan para crear, actualizar y manipular bases de datos. Estos conceptos son bastante amplios por eso será mejor que vayamos por partes.

- ✔ **COMANDOS:** Van a ser las instrucciones que se pueden crear en SQL. Se pueden distinguir en tres grupos que veremos con más detenimiento a lo largo de las siguientes unidades:
 - ➡ De definición de datos (DDL, Data Definition Language), que permiten crear y definir nuevas bases de datos, tablas, campos, etc.
 - ➡ De manipulación de datos (DML, Data Manipulation Language), que permiten generar consultas para ordenar, filtrar y extraer datos de la base de datos.
 - ➡ De control y seguridad de datos (DCL, Data Control Language), que administran los derechos y restricciones de los usuarios.
- ✔ **CLÁUSULAS:** Llamadas también condiciones o criterios, son palabras especiales que permiten modificar el funcionamiento de un comando.
- ✔ **OPERADORES:** Permiten crear expresiones complejas. Pueden ser aritméticos (+, -, *, /, ...) o lógicos (<, >, <=, >=, And, Or, etc.).
- ✔ **FUNCIONES:** Para conseguir valores complejos. Por ejemplo, la función promedio para obtener la media de un salario.
- ✔ **LITERALES:** Les podemos llamar también constantes y serán valores concretos, como por ejemplo un número, una fecha, un conjunto de caracteres, etc.

Y tendremos que seguir unas normas sencillas pero primordiales:

- ✔ Todas las instrucciones **terminan con un signo de punto y coma**.
- ✔ No se distingue entre mayúsculas y minúsculas.
- ✔ Cualquier comando puede ser partido con saltos de línea o espacios para facilitar su lectura y comprensión.
- ✔ Los comentarios comienzan por /* y terminan con */ (excepto en algunos SGBD).

Juan le ha dicho a Ana que es hora de ponerse a trabajar con la aplicación. Para aprender mejor le ha pedido permiso a Juan para instalar Oracle en su ordenador y así ir probando todo sobre la marcha para no cometer errores. El SQL estándar y el SQL de Oracle son bastante parecidos, pero con algunas diferencias.

Debes conocer

En el siguiente documento encontraras aquellos comandos, cláusulas, operadores y funciones más generales con las que vamos a trabajar a lo largo del curso:

Para saber más

Otro Sistema Gestor de Base de Datos muy utilizado en algunos entornos como el de desarrollo web es MySQL. Sería interesante que lo conocieras y supieras instalarlo:

[Manual MySQL.](#)

Otra página recomendable donde puedes aprender MySQL desde cero es la siguiente:

[MySQL con Clase.](#)

11.- Lenguaje de descripción de datos (DDL).

Caso práctico



[Ministerio de Educación](#) (Uso educativo no)

Ana y Juan han realizado concienzudamente el diseño de las tablas necesarias para la base de datos de la aplicación en la que están trabajando.



[Ministerio de Educación](#) (Uso educativo no)

También se han decantado por el sistema gestor de bases de datos a utilizar. Emplearán un sistema gestor de bases de datos relacional. Una vez instalado el sistema gestor, tendrán que programar los accesos a la base de datos para guardar los datos, recuperarlos, realizar las consultas para los informes y documentos que sean necesarios, etc.

Ana está creando la primeras tablas de la base de datos. Una de las principales es USUARIO, aunque también tendrá que crear la de PARTIDAS y JUEGOS.

La primera fase del trabajo con cualquier base de datos comienza con sentencias DDL, puesto que antes de poder almacenar y recuperar información debemos definir las estructuras donde almacenar la información. Las estructuras básicas con las que trabaja SQL son las tablas.

Conocer el Lenguaje de Definición de Datos (DDL) es imprescindible para crear, modificar y eliminar objetos de la base de datos (es decir, los metadatos). En el mercado hay suficientes aplicaciones y asistentes que nos facilitan esta labor, a través de una interfaz visual que nos oculta el lenguaje SQL y en los cuales nos limitamos a poner nombres a los campos, elegir el tipo de datos y activar una serie de propiedades.

Es cierto que estas herramientas nos facilitan el trabajo, pero resulta imprescindible comprender y conocer en profundidad el lenguaje, ya que nos veremos en muchas situaciones donde necesitaremos crear un objeto, modificarlo o eliminarlo sin depender de esas herramientas visuales.

En Oracle, cada usuario de una base de datos tiene un esquema, que tendrá el mismo nombre que el usuario con el que se ha accedido y sirve para almacenar los objetos que posea ese usuario.

¿De qué objetos estamos hablando? Éstos podrán ser tablas, vistas, índices u otros objetos relacionados con la definición de la base de datos. ¿Y quién puede crear y manipularlos? En principio el usuario propietario (el que los creó) y los administradores de la base de datos. Más adelante veremos que podemos modificar los privilegios de los objetos para permitir el acceso a otros usuarios.

Las instrucciones DDL generan acciones que no se pueden deshacer, por eso es conveniente usarlas con precaución y tener copias de seguridad cuando manipulamos la base de datos.

Para saber más

Si quieres saber un poco más sobre el Lenguaje de Definición de Datos, puedes visitar la Wikipedia, aquí tienes el enlace:

[Lenguaje de Definición de Datos](#)

11.1.- Creación de bases de datos. Objetos de la base de datos.

Básicamente, la creación de la base de datos consiste en crear las tablas que la componen. Aunque antes de esto tendríamos que definir un espacio de nombres separado para cada conjunto de tablas. Es lo que antes hemos llamado **esquemas** o **usuarios**.



Pilar Ramírez. (Uso educativo no.)

Crear una base de datos implica indicar los archivos y ubicaciones que se van a utilizar además de otras indicaciones técnicas y administrativas. Es obvio que todo esto sólo lo puede realizar si se tiene privilegio de Administrador.

Con el estándar de SQL la instrucción a usar sería **Create Database**, pero cada SGBD tiene un procedimiento para crear las bases de datos. Crearíamos una base de datos con el nombre que se indique a continuación.

```
CREATE DATABASE NombredemiBasedeDatos;
```

Por ejemplo, a la base de datos que están creando Juan y Ana se le va a llamar RyMjuegos, entonces nos quedaría:

```
CREATE DATABASE RyMjuegos;
```

Hemos estado hablando de objetos de la base de datos, ahora veremos a qué nos referimos.

Según los estándares, **una base de datos es un conjunto de objetos que nos servirán para gestionar los datos**. Estos objetos están contenidos en esquemas y éstos a su vez suelen estar asociados a un usuario. De ahí que antes dijéramos que cada base de datos tiene un esquema que está asociado a un usuario.

En oracle la sentencia "create database" tiene un sentido distinto a otros SGBD como MySQL.

Nosotros trabajaremos en oracle creando las tablas en el esquema del usuario creado previamente, es decir, no utilizaremos la sentencia "create database"

Para saber más

Si quieres aprender a crear bases de datos con MySQL, aquí puedes aprender:

[MySQL: creación de bases de datos.](#)

11.2.- Creación de tablas.

¿Qué necesitamos para poder guardar los datos? Lo primero será definir los objetos donde vamos a agrupar esos datos. Los objetos básicos con los que trabaja SQL son las tablas, que como ya sabemos es un conjunto de filas y columnas cuya intersección se llama celda. Es ahí donde se almacenarán los elementos de información, los datos que queremos recoger.

Código	Nombre	Cod_Juego
1	PARTIDA01	3
2	PARTIDA02	2
3	PARTIDA03	4
4	PARTIDA04	3
5	PARTIDA05	1

Pilar Ramírez. (Uso educativo no)

Antes de crear la tabla es conveniente tener a mano la siguiente información que se obtiene en la fase de diseño lógico de la BD

- ✓ **Qué nombre** le vamos a dar a la **tabla**.
- ✓ **Qué nombre** le vamos a dar a cada una de las **columnas**.
- ✓ **Qué tipo y tamaño de datos** vamos a almacenar en cada columna.
- ✓ **Qué restricciones** tenemos sobre los datos.
- ✓ Alguna otra **información adicional** que necesitemos.

Y debemos tener en cuenta otras **reglas** que se deben cumplir **para los nombres de las tablas**:

- ✓ No podemos tener nombres de tablas duplicados en un mismo esquema (usuario).
- ✓ Deben comenzar por un carácter alfabético.
- ✓ Su longitud máxima es de 30 caracteres.
- ✓ Solo se permiten letras del alfabeto inglés, dígitos o el signo de guión bajo.
- ✓ No puede coincidir con las palabras reservadas de SQL (por ejemplo, no podemos llamar a una tabla WHERE).
- ✓ No se distingue entre mayúsculas y minúsculas.
- ✓ En el caso de que el nombre tenga espacios en blanco o caracteres nacionales (permitido sólo en algunas bases de datos), entonces se suele entrecomillar con comillas dobles. En el estándar [SQL99](#) (respetado por Oracle) se pueden utilizar comillas dobles al poner el nombre de la tabla a fin de hacerla sensible a las mayúsculas (se diferenciará entre "USUARIOS"y "Usuarios").

La sintaxis básica del comando que permite crear una tabla es la siguiente:

Sintaxis básica:

```
CREATE TABLE [esquema.]nombreDeTabla (  
    columna1 Tipo_Dato [restricciones],  
    columna2 Tipo_Dato [restricciones],  
    ...  
    columnaN Tipo_Dato [restricciones]  
);
```

Ejemplo 1: Tabla simple

```
CREATE TABLE usuarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Ejemplo 2: Tabla con clave foránea

```
CREATE TABLE pedidos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  usuario_id INT NOT NULL,  
  total DECIMAL(10,2) NOT NULL,  
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE  
);
```

Verificar las tablas creadas:

```
SHOW TABLES;  
DESCRIBE usuarios;
```

donde:

- ✔ columna1, columna2, ..., columnaN son los nombres de las columna que contendrá la tabla.
- ✔ Tipo_Dato indica el tipo de dato de cada columna.

Ana va a crear la primera tabla llamada USUARIOS con un solo campo de tipo VARCHAR:

```
CREATE TABLE USUARIOS (Nombre VARCHAR(25));
```

Recuerda que solo podrás crear tablas si posees los permisos necesarios para ello.

Debes conocer

Durante nuestro aprendizaje vamos a tener que crear muchas tablas, para ello necesitaremos manejar los tipos de datos que utiliza MySQL. En el siguiente enlace tienes una relación de estos tipos y su descripción.

[Tipos de datos en MySql.](#)

Para saber más

MySQL trabaja con otros tipos de datos. Si quieres conocerlos puedes entrar en este enlace.

[Tipos de datos en MySQL.](#)

Autoevaluación

Señala cuales de las siguientes afirmaciones sobre los nombres de las tablas son ciertas:

- ☐ Puede haber nombres de tablas duplicados en la misma base de datos.

- ☐ Su longitud máxima es de 30 caracteres.

- ☐ La tabla JUEGOS es la misma que la tabla Juegos.

- ☐ No puede coincidir con las palabras reservadas de SQL.

Mostrar retroalimentación

Solución

1. Incorrecto
2. Correcto
3. Correcto
4. Correcto

11.3.- Restricciones.

Hay veces que necesitamos que un dato se incluya en una tabla de manera obligatoria, otras veces necesitaremos definir uno de los campos como llave primaria o ajena. Todo esto podremos hacerlo cuando definamos la tabla, además de otras opciones.



[Anonymous](#) (Open Clip Art .CC)

Una **restricción** es una condición que una o varias columnas deben cumplir obligatoriamente.

Cada restricción que creemos llevará un nombre, si no se lo ponemos nosotros lo hará Oracle o el SGBD que estemos utilizando. Es conveniente que le pongamos un nombre que nos ayude a identificarla y que sea único para cada esquema (usuario). Es buena idea incluir de algún modo el nombre de la tabla, los campos involucrados y el tipo de restricción en el nombre de la misma. La sintaxis en SQL estándar es la siguiente:



Crear Tabla en MySQL con Restricciones

Sintaxis:

```
CREATE TABLE [nombre_de_la_tabla] (  
    [columna1] [tipo_dato] [restricciones],  
    [columna2] [tipo_dato] [restricciones],  
    [columnaN] [tipo_dato] [restricciones],  
    [FOREIGN KEY] ([columna_foranea]) REFERENCES [tabla_referenciada]([columna]) [ON DELETE CASCADE]  
);
```



Ejemplo práctico:

```
CREATE TABLE [empleados] (  
    [id] INT AUTO_INCREMENT [PRIMARY KEY],  
    [nombre] VARCHAR(100) [NOT NULL],  
    [email] VARCHAR(150) [UNIQUE] [NOT NULL],  
    [salario] DECIMAL(10,2) [CHECK (salario > 0)],  
    [departamento_id] INT,  
    [FOREIGN KEY] ([departamento_id]) REFERENCES [departamentos]([id]) [ON DELETE CASCADE]  
);
```

Los corchetes, caracteres [y], se utilizan en informática en los formatos de la sintaxis de los lenguajes para especificar opcionalidad, es decir, está indicando que lo contiene se puede utilizar o no.

Veamos un ejemplo:

```
CREATE TABLE USUARIOS (  
  Login VARCHAR(15) PRIMARY KEY,  
  Password VARCHAR(8) NOT NULL,  
  Fecha_Ingreso DATE DEFAULT (CURRENT_DATE));
```

Otra opción es definir las columnas de la tabla y después especificar las restricciones, de este modo podrás referir varias columnas en una única restricción.

En los siguientes apartados veremos cada una de las restricciones, su significado y su uso.

Recomendación

MySQL recomienda la siguiente convención a la hora de poner nombres a las restricciones:

- ✔ Tres letras para el nombre de la tabla.
- ✔ Carácter de subrayado.
- ✔ Tres letras con la columna afectada por la restricción.
- ✔ Carácter de subrayado.
- ✔ Dos letras con la abreviatura del tipo de restricción. La abreviatura puede ser:
 - ➡ **PK** = Primary Key (Clave primaria).
 - ➡ **FK** = Foreign Key (Clave foránea).
 - ➡ **NN** = Not Null (No permitir valores nulos).
 - ➡ **UQ** = Unique (Valores únicos en la columna).
 - ➡ **CK** = Check (Condición de validación - Disponible en MySQL 8.0+).
 - ➡ **DF** = Default (Valor por defecto).

11.3.1.- Restricción NOT NULL.

Con esta restricción obligaremos a que esa columna tenga un valor o lo que es lo mismo, prohíbe los valores nulos para una columna en una determinada tabla.

Podremos ponerlo cuando creamos o modificamos el campo añadiendo la palabra **NOT NULL** después de poner el tipo de dato.

Si en la tabla **USUARIOS** queremos que el campo "F_Nacimiento" sea obligatorio ponerlo, nos quedaría así:



Pilar Ramírez. (Uso educativo nc)

```
CREATE TABLE USUARIOS (  
    F_Nacimiento DATE,  
    CONSTRAINT Usu_Fnac_NN CHECK (F_Nacimiento IS NOT NULL)  
);
```

Nota: En MySQL, la restricción **CHECK** se ignora en versiones anteriores a MySQL 8.0. En esas versiones, **CHECK** no tiene efecto, por lo que la siguiente forma es más recomendable.

```
CREATE TABLE USUARIOS (  
    F_Nacimiento DATE NOT NULL);
```

Debemos tener cuidado con los valores nulos en las operaciones, ya que $1 * \text{NULL}$ es igual a **NULL**.

Autoevaluación

Si queremos que un campo no admita valores nulos, al crear la tabla pondremos después del nombre del campo y del tipo de datos:

- ☐ **NULL.**
- ☐ **VARCHAR.**
- ☐ **NOT NULL.**

No es correcto, de este modo estamos indicando que el valor es nulo.

Incorrecto, **VARCHAR** es un tipo de datos.

Estupendo, sigue así.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

11.3.2.- Restricción UNIQUE.

Restricción UNIQUE en MySQL

Habr  ocasiones en las que necesitemos evitar la repetici n de valores en una columna. En estos casos, utilizaremos la restricci n **UNIQUE**, que garantiza que los valores de una columna o combinaci n de columnas sean  nicos dentro de la tabla.

En **MySQL**, cuando se define una restricci n **UNIQUE**, el sistema autom ticamente crea un ** ndice  nico** en la columna afectada. Sin embargo, si la restricci n **UNIQUE** se elimina, el  ndice permanece y debe eliminarse manualmente si ya no es necesario.

Existen dos formas de definir la restricci n **UNIQUE** en MySQL. Supongamos que el campo **Login** de nuestra tabla debe ser  nico. Podemos definirlo de las siguientes maneras:



[user9](#) (Open Clip Art .CC)

1 Definir **UNIQUE** directamente en la columna (Forma Simple)

En esta forma, la restricci n **UNIQUE** se especifica junto con el tipo de dato de la columna:

```
CREATE TABLE USUARIOS (  
    Login VARCHAR(15) UNIQUE  
);
```

2 Definir **UNIQUE** con **CONSTRAINT** (Forma Expl cita)

Tambi n podemos asignarle un nombre a la restricci n usando **CONSTRAINT**, lo que facilita su administraci n:

```
CREATE TABLE USUARIOS (  
    Login VARCHAR(15),  
    CONSTRAINT Usu_Log_UQ UNIQUE (Login)  
);
```

Eliminaci n de la restricci n **UNIQUE** en MySQL

Si en alg n momento necesitamos eliminar la restricci n **UNIQUE**, podemos hacerlo de la siguiente manera:

```
ALTER TABLE USUARIOS DROP INDEX Usu_Log_UQ;
```

Nota: A diferencia de otros sistemas, MySQL no elimina autom ticamente el  ndice asociado a **UNIQUE** al quitar la restricci n, por lo que es necesario eliminarlo manualmente.

Resumen:

- **UNIQUE** en MySQL impide valores duplicados en una columna o combinaci n de columnas.

- **MySQL crea un índice automáticamente** al definir **UNIQUE**, pero no lo borra automáticamente al eliminar la restricción.
- Dos formas de definir **UNIQUE**: directamente en la columna o con **CONSTRAINT**.
- **Si eliminas UNIQUE, recuerda eliminar el índice manualmente** con **DROP INDEX**.

Este comportamiento es importante al diseñar bases de datos en **MySQL** para evitar duplicados de manera eficiente.

También podemos poner esta restricción a varios campos a la vez, por ejemplo, si queremos que Login y correo electrónico sean únicos podemos ponerlo así:

```
CREATE TABLE USUARIOS (  
    Login VARCHAR2 (25),  
    Correo VARCHAR2 (25),  
    CONSTRAINT Usuario_UK UNIQUE (Login, Correo));
```

Si te fijas, detrás del tipo de datos de Correo hay una coma, eso es así porque la restricción es independiente de ese campo y común a varios. Por eso después de **UNIQUE** hemos puesto entre paréntesis los nombres de los campos a los que afecta la restricción.

11.3.3.- Restricción PRIMARY KEY.

En el modelo relacional las tablas deben tener una clave primaria. Es evidente que cuando creamos la tabla tendremos que indicar a quién corresponde.

Sólo puede haber una clave primaria por tabla pero ésta puede estar formada por varios campos. Dicha clave podrá ser referenciada como clave ajena en otras tablas.

La clave primaria hace que los campos que forman sean **NOT NULL** y que los valores de los campos sean de tipo **UNIQUE**.

Veamos como quedaría si la clave fuese el campo Login:

- ✓ Si la clave la forma un único campo:

```
CREATE TABLE USUARIOS (  
    Login VARCHAR(25) PRIMARY KEY  
);
```

- ✓ O bien poniendo un nombre a la restricción:

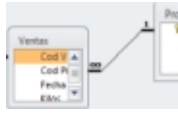
```
CREATE TABLE USUARIOS (  
    Login VARCHAR(25),  
    CONSTRAINT Usu_log_PK PRIMARY KEY (Login)  
);
```

- ✓ Si la clave está formada por más de un campo, por ejemplo Nombre, Apellidos y Fecha de Nacimiento, entonces hay que utilizar este formato, después de la definición de todos los campos y antes del paréntesis de cierre de la sentencia:

```
CREATE TABLE USUARIOS (  
    Nombre VARCHAR(25),  
    Apellidos VARCHAR(30),  
    F_Nacimiento DATE,  
    CONSTRAINT Usu_PK PRIMARY KEY (Nombre, Apellidos, F_Nacimiento)  
);
```

11.3.4.- Restricción REFERENCES. FOREIGN KEY.

Ya vimos que las claves ajenas, secundarias o foráneas son campos de una tabla que se relacionan con la clave primaria (o incluso con una clave candidata) de otra tabla.



Pilar Ramírez. (Uso educativo
nc)

Cuando creamos la tabla, tendremos que indicar de alguna forma qué campo es clave ajena. En MySQL, lo haremos utilizando la cláusula **FOREIGN KEY** y "haciendo referencia" a la tabla y los campos de los que procede.

En nuestra tabla, vamos a tener una clave ajena procedente de la tabla **PARTIDAS**, que será su **Cod_Partida**. Por lo tanto, debemos hacer referencia a este campo:

```
CREATE TABLE USUARIOS (  
    Cod_Partida INT(8),  
    CONSTRAINT Cod_Part_FK FOREIGN KEY (Cod_Partida) REFERENCES PARTIDAS(Cod_Partida)  
);
```

Si el campo al que hace referencia es clave primaria en su tabla, no es necesario indicar el nombre del campo en MySQL:

```
CREATE TABLE USUARIOS (  
    Cod_Partida INT(8),  
    CONSTRAINT Cod_Part_FK FOREIGN KEY (Cod_Partida) REFERENCES PARTIDAS  
);
```

Si la definición de la clave ajena se coloca al final de la tabla, en MySQL debemos utilizar **FOREIGN KEY** para especificar a qué campo nos referimos. Veamos un ejemplo en el que la clave ajena está compuesta por **Cod_Partida** y **F_Partida** de la tabla **PARTIDAS**:

```
CREATE TABLE USUARIOS (  
    Cod_Partida INT(8),  
    F_Partida DATE,  
    CONSTRAINT Partida_Cod_F_FK FOREIGN KEY (Cod_Partida, F_Partida) REFERENCES PARTIDAS(Cod_Partida, F_P  
);
```



Al relacionar campos en MySQL, es importante asegurarse de que los valores insertados en el campo que actúa como clave foránea (en la tabla secundaria) ya existan previamente en la tabla de origen, donde ese campo es clave primaria o candidata. En nuestro ejemplo, cualquier código de partida que incluyamos en la tabla **USUARIOS** debe existir previamente en la tabla **PARTIDAS**. **A esto se le llama Integridad Referencial.**

Este mecanismo puede generar algunos errores si no se respetan las reglas de integridad, por ejemplo:

- ✔ Si hacemos referencia a una tabla que no está creada: MySQL buscará la tabla referenciada y, al no encontrarla, generará un error. Para evitar esto, debemos asegurarnos de crear primero las tablas que no tengan claves ajenas.
- ✔ Si queremos borrar tablas con claves ajenas: En MySQL, primero debemos eliminar las tablas que contienen claves foráneas antes de eliminar la tabla referenciada.

Para gestionar la eliminación de registros en una relación con clave foránea, MySQL nos ofrece varias opciones adicionales tras la cláusula **REFERENCES**:

- ✔ **ON DELETE CASCADE**: Permite borrar todos los registros cuya clave ajena coincida con la clave primaria del registro eliminado.
- ✔ **ON DELETE SET NULL**: Coloca el valor **NULL** en todas las claves ajenas relacionadas con la clave eliminada.
- ✔ **ON DELETE DEFAULT 'xxx'**: Coloca un valor por defecto en todas las claves ajenas relacionadas con la clave eliminada.

Estas opciones también pueden aplicarse en la operación de modificación de registros, especificando **ON UPDATE** en lugar de **ON DELETE**.

11.3.5.- Restricción DEFAULT Y VALIDACIÓN.

A veces es muy tedioso insertar siempre lo mismo en un campo. Imagínate que casi todos los jugadores fuesen de España y tenemos un campo **País**. ¿No sería cómodo asignarle un valor por defecto? Eso es lo que hace la restricción **DEFAULT**.

En nuestro ejemplo, vamos a añadir a la tabla **USUARIOS** el campo **País** y le daremos por defecto el valor "España".

```
CREATE TABLE USUARIOS (  
    País VARCHAR(20) DEFAULT 'España'  
);
```

En las especificaciones de **DEFAULT**, podemos añadir distintas expresiones, como constantes, funciones SQL y variables.

Si queremos incluir en un campo la fecha actual, independientemente del día en el que estemos, podemos utilizar la función **CURRENT_TIMESTAMP** como valor por defecto:

```
CREATE TABLE USUARIOS (  
    Fecha_ingreso TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

También podemos necesitar que se compruebe que los valores introducidos en un campo sean adecuados. Para ello utilizaremos **CHECK**.

Esta restricción verifica que se cumpla una condición determinada al rellenar una columna. La condición puede construirse con valores de esa misma tabla.

Por ejemplo, si en la tabla **USUARIOS** tenemos el campo **Crédito** y queremos que solo pueda tener valores entre 0 y 2000, lo especificaríamos así:

```
CREATE TABLE USUARIOS (  
    Credito INT CHECK (Credito BETWEEN 0 AND 2000)  
);
```

Una misma columna puede tener varias restricciones **CHECK** aplicadas. Para ello, podemos definir varias restricciones con **CONSTRAINT** y separarlas por comas.

Debes conocer

Si queremos obtener una descripción de una tabla en MySQL, podemos utilizar el comando **DESCRIBE** o su equivalente **SHOW CREATE TABLE**.

Ejemplo con DESCRIBE:

```
DESCRIBE usuarios;
```

Ejemplo con SHOW CREATE TABLE:

```
SHOW CREATE TABLE usuarios;
```

En el siguiente enlace tienes información sobre los comandos básicos de MySQL:

[Documentación Oficial de MySQL](#)

Autoevaluación

Relaciona estos términos utilizados para las restricciones en la creación de tablas con su significado o función:

Términos.	Relación.	Función.
CHECK	☐	1. Comprueba que los valores que se introducen son los adecuados para un campo.
DEFAULT	☐	2. Designa a un campo como clave ajena.
PRIMARY KEY	☐	3. Impide que un campo pueda contener valores nulos
FOREIGN KEY	☐	4. Impide que se repitan valores para un campo.
NOT NULL	☐	5. Designa a un campo como clave principal.
UNIQUE	☐	6. Incluye un valor en un campo de forma predeterminada.

Enviar

Estupendo, ya tienes todas las herramientas para crear una tabla.

11.4.- Eliminación de tablas.

Cuando una tabla ya no es útil y no la necesitamos, es mejor borrarla. De este modo, no ocupará espacio y podremos utilizar su nombre en otra ocasión.

[warszawianka](#) (Open Clip Art .CC)

Para eliminar una tabla en MySQL, utilizamos el comando **DROP TABLE**:

```
DROP TABLE NombreTabla;
```



Esta instrucción borrará la tabla de la base de datos, incluyendo sus datos (filas). También se eliminará toda la información que existiera de esa tabla en el diccionario de datos.

Si una tabla tiene claves foráneas en otras tablas, es necesario eliminarlas primero o usar la opción **ON DELETE CASCADE** en la definición de la clave foránea.

Vamos a eliminar la tabla con la que hemos estado trabajando:

```
DROP TABLE USUARIOS;
```

⚠ **Ten cuidado al utilizar este comando**, ya que el borrado de una tabla es irreversible y no hay una petición de confirmación antes de ejecutarse.

Al borrar una tabla:

- ✔ Desaparecen todos sus datos.
- ✔ Cualquier vista asociada a esa tabla seguirá existiendo, pero ya no funcionará.

MySQL dispone de la orden **TRUNCATE TABLE**, que permite eliminar los datos (filas) de una tabla sin eliminar su estructura.

```
TRUNCATE TABLE NombreTabla;
```

Y recuerda que solo podrás borrar aquellas tablas sobre las que tengas permiso de eliminación.

11.5.- Modificación de tablas (I).

Después de crear una tabla, puede ser necesario añadir un campo o restricción, eliminar una columna, o modificar el nombre de algún campo. En MySQL, esto se realiza mediante el comando `ALTER TABLE`.

- ✓ **Si queremos cambiar el nombre de una tabla:**

```
RENAME TABLE NombreViejo TO NombreNuevo;
```

- ✓ **Si queremos añadir columnas a una tabla:** las columnas se añadirán al final de la tabla.

```
ALTER TABLE NombreTabla ADD  
  ( ColumnaNueva1 Tipo_Dato [Propiedades]  
    [, ColumnaNueva2 Tipo_Dato [Propiedades]  
    ... ] );
```

- ✓ **Si queremos eliminar columnas de una tabla:** se eliminará la columna indicada sin posibilidad de deshacer la acción. No se puede eliminar una columna si es la única que forma la tabla; en ese caso, se debe borrar la tabla directamente.

```
ALTER TABLE NombreTabla DROP COLUMN Columna1;
```

- ✓ **Si queremos modificar columnas de una tabla:** podemos modificar el tipo de datos y las propiedades de una columna. Algunos cambios son posibles solo si la tabla no contiene datos.

```
ALTER TABLE NombreTabla MODIFY  
  (Columna1 Tipo_Dato [Propiedades]  
    [, Columna2 Tipo_Dato [Propiedades] ...]);
```

- ✓ **Si queremos renombrar columnas de una tabla:**

```
ALTER TABLE NombreTabla RENAME COLUMN NombreAntiguo TO NombreNuevo;
```

Tenemos la siguiente tabla creada:

```
CREATE TABLE USUARIOS (  
  Credito INT CHECK (Credito BETWEEN 0 AND 2000));
```

Nos gustaría incluir una nueva columna llamada User que será tipo texto y clave primaria:


```
ALTER TABLE USUARIOS ADD  
  (User VARCHAR(10) PRIMARY KEY);
```

Nos damos cuenta de que ese campo se llamaba Login y no User, así que lo cambiamos:

```
ALTER TABLE USUARIOS RENAME COLUMN User TO Login;
```

Ejercicio resuelto

Tenemos creada la siguiente tabla:

```
CREATE TABLE EMPLEADOS (  
  Cod_Cliente VARCHAR(5) PRIMARY KEY,  
  Nombre VARCHAR(10),  
  Apellidos VARCHAR(25),  
  Sueldo INT);
```

Ahora queremos poner una restricción a **Sueldo** para que tome valores entre 1000 y 1200.
¿Cómo lo harías?

Mostrar retroalimentación

```
ALTER TABLE EMPLEADOS MODIFY Sueldo INT CHECK (Sueldo BETWEEN 1000 AND 1200);
```

11.5.1.- Modificación de tablas (II).

Utilizando el comando `ALTER TABLE`, podemos modificar las restricciones de una tabla o bien eliminarlas cuando sea necesario.

✔ Si queremos borrar restricciones:

```
ALTER TABLE NombreTabla DROP CONSTRAINT NombreRestriccion;
```

✔ Si queremos modificar el nombre de una restricción:

```
ALTER TABLE NombreTabla RENAME CONSTRAINT NombreViejo TO NombreNuevo;
```

✔ Si queremos activar o desactivar restricciones:

A veces es conveniente desactivar temporalmente una restricción para hacer pruebas o porque necesitemos saltarnos esa regla. En MySQL, se puede hacer desactivando las verificaciones de claves foráneas con el siguiente comando:

```
SET FOREIGN_KEY_CHECKS = 0;
```

Para volver a activarlas:

```
SET FOREIGN_KEY_CHECKS = 1;
```

Sin embargo, si deseamos eliminar una restricción específica de una clave foránea, se hace con:

```
ALTER TABLE NombreTabla DROP FOREIGN KEY NombreRestriccion;
```

Para activar una clave foránea nuevamente, en MySQL sería necesario volver a crearla con `ADD CONSTRAINT`, como en el siguiente ejemplo:

```
ALTER TABLE NombreTabla ADD CONSTRAINT NombreRestriccion  
FOREIGN KEY (Columna) REFERENCES TablaReferenciada(ColumnaReferenciada);
```

Debes conocer

Puede ocurrir que no hayamos puesto nombre a las restricciones o bien que lo hiciéramos, pero no lo recordemos. Para ello, en MySQL podemos consultar las restricciones de una tabla con el siguiente comando:

```
SELECT * FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS
WHERE TABLE_NAME = 'NombreTabla';
```

En este resultado se mostrará el tipo de restricción aplicada a la tabla, como claves primarias, claves foráneas y restricciones `UNIQUE`.

11.6.- Creación y eliminación de índices

Sabemos que crear índices ayuda a la localización más rápida de la información contenida en las tablas. Ahora aprenderemos a crearlos y eliminarlos:

```
CREATE INDEX NombreIndice ON NombreTabla (Columna1 [, Columna2 ...]);
```

No es aconsejable que utilices índices sobre campos de tablas pequeñas o que se actualicen con mucha frecuencia. Tampoco es conveniente si esos campos no se usan en consultas de manera frecuente o en expresiones.

El diseño de índices es un tema bastante complejo para los Administradores de Bases de Datos, ya que una mala elección ocasiona ineficiencia y tiempos de espera elevados. Un uso excesivo de ellos puede dejar a la Base de Datos colgada simplemente con insertar alguna fila.

Para eliminar un índice es suficiente con poner la instrucción:

```
DROP INDEX NombreIndice ON NombreTabla;
```

La mayoría de los índices se crean de manera implícita cuando ponemos las restricciones **PRIMARY KEY**, **FOREIGN KEY** o **UNIQUE**.

Ejercicio resuelto

Tenemos creada la siguiente tabla:

```
CREATE TABLE EMPLEADOS (  
    Cod_Cliente VARCHAR(5) PRIMARY KEY,  
    Nombre VARCHAR(10),  
    Apellidos VARCHAR(25),  
    Sueldo INT);
```

Crea un índice con el campo Apellidos, luego elimínalo.

[Mostrar retroalimentación](#)

```
CREATE INDEX miIndice ON EMPLEADOS (Apellidos);  
DROP INDEX miIndice ON EMPLEADOS;
```

12.- Lenguaje de control de datos (DCL).

Caso práctico



[Ministerio de Educación](#) (Uso educativo nc)

Juan cree que será necesario conocer quiénes acceden a la base de datos para poder crearles sus contraseñas y darles los permisos necesarios, de manera que la administración de la base quede en manos de quien corresponde, y el sistema sea seguro. No quiere ni imaginarse que quedara algún cabo suelto, y cualquier usuario con algo de conocimientos pudiera acceder sin consentimiento y con los permisos suficientes, como para manipular los datos a su antojo.

Ya hemos visto que necesitamos una cuenta de usuario para acceder a los datos de una base de datos. Las claves de acceso se establecen cuando se crea el usuario y pueden ser modificadas por el Administrador o por el propietario de dicha clave. La Base de Datos almacena encriptadas las claves en una tabla del sistema llamada `mysql.user`.

¿Cómo se crean los usuarios? La sintaxis en MySQL es:



[Leandro Sciola](#) (Open Clip Art .CC)

```
CREATE USER 'NombreUsuario'@'localhost'  
IDENTIFIED BY 'ClaveAcceso';
```

donde:

- ✓ **CREATE USER:** crea un usuario que será identificado en el sistema.
- ✓ **IDENTIFIED BY:** establece una clave de acceso para el usuario.
- ✓ **'NombreUsuario'@'localhost':** define el nombre del usuario y el host desde el cual puede conectarse.

Recuerda que para crear usuarios debes tener privilegios de Administrador.

Para ver todos los usuarios creados en MySQL se usa:

```
SELECT User, Host FROM mysql.user;
```

Practiquemos un poco con este comando. Creemos una cuenta de usuario limitado, que no tenga derecho ni a guardar datos ni a crear objetos. Más tarde le daremos permisos:

```
CREATE USER 'UsuarioLimitado'@'localhost'  
IDENTIFIED BY 'passworddemiusuariolimitado';
```

Podemos modificar usuarios mediante el comando `ALTER USER`, cuya sintaxis es la siguiente:

```
ALTER USER 'NombreUsuario'@'localhost'  
IDENTIFIED BY 'NuevaClaveAcceso';
```

Un usuario sin privilegios de Administrador únicamente podrá cambiar su clave de acceso.

Para eliminar o borrar un usuario utilizamos el comando **DROP USER** con la siguiente sintaxis:

```
DROP USER 'NombreUsuario'@'localhost';
```

Si el usuario tiene objetos en la base de datos, es recomendable eliminarlos antes de borrar la cuenta para evitar errores.

12.1.- Permisos (I).

Ningún usuario puede llevar a cabo una operación si antes no se le ha concedido el permiso para ello. En el apartado anterior hemos creado un usuario para iniciar sesión, pero si con él intentáramos crear una tabla veríamos que no tenemos permisos suficientes para ello.

Para poder acceder a los objetos de una base de datos necesitas tener privilegios (permisos). Éstos se pueden agrupar formando **roles**, lo que simplificará la administración. Los roles pueden activarse, desactivarse o protegerse con una clave. Mediante los roles podemos gestionar los comandos que pueden utilizar los usuarios. Un permiso se puede asignar a un usuario o a un rol.

Un privilegio o permiso se concede con el comando **GRANT** (conceder).

Si se dan privilegios **sobre los objetos**:

```
GRANT {privilegio_objeto [, privilegio_objeto]...|ALL}
  ON NombreTabla
  TO {usuario1 | rol1} [{usuario2 | rol2} ...]
  [WITH GRANT OPTION];
```

donde:

- ✔ **ON** especifica el objeto sobre el que se conceden los privilegios.
- ✔ **TO** señala a los usuarios o roles a los que se conceden privilegios.
- ✔ **ALL** concede todos los privilegios sobre el objeto especificado.
- ✔ **[WITH GRANT OPTION]** permite que el receptor del privilegio se lo pueda conceder a otros.

En el siguiente ejemplo Juan ha accedido a la base de datos y ejecuta los siguientes comandos:

- ✔ **GRANT INSERT ON Usuarios TO 'Ana'@'localhost';** (permitirá a Ana insertar datos en la tabla Usuarios)
- ✔ **GRANT ALL ON Partidas TO 'Ana'@'localhost';** (Juan concede todos los privilegios sobre la tabla Partidas a Ana)

Los privilegios **de sistema** son los que dan derecho a ejecutar comandos SQL o acciones sobre objetos de un tipo especificado. Existen gran cantidad de privilegios distintos.

La sintaxis para dar este tipo de privilegios la tienes aquí:

```
GRANT {privilegio1 | rol1} [, privilegio2 | rol2]...
  TO {usuario1 | rol1} [, usuario2 | rol2] ...
  [WITH ADMIN OPTION];
```

- ✔ **TO** señala a los usuarios o roles a los que se conceden privilegios.
- ✔ **WITH ADMIN OPTION** es una opción que permite al receptor de esos privilegios que pueda conceder esos mismos privilegios a otros usuarios o roles.

Veamos algunos ejemplos:


```
GRANT SELECT ON BaseDeDatos.* TO 'Ana'@'localhost';
```

Concede a Ana el permiso de lectura en todas las tablas de la base de datos.

```
GRANT DROP USER ON *.* TO 'Ana'@'localhost' WITH ADMIN OPTION;
```

Concede a Ana el privilegio de borrar usuarios y que ésta pueda conceder el mismo privilegio a otros.

Para saber más

Si quieres conocer más sobre permisos y objetos sobre los que se conceden privilegios, visita este enlace:

[Gestión de privilegios y recursos en MySQL.](#)

12.1.1.- Permisos (II).

Hasta ahora hemos aprendido a conceder permisos o privilegios. Será importante aprender a retirarlos:

Con el comando **REVOKE** se retiran los privilegios:

✓ Sobre **objetos**:

```
REVOKE {privilegio_objeto [, privilegio_objeto]...|ALL|PRIVILEGES}
ON [usuario.]objeto
FROM {usuario|rol|PUBLIC} [{usuario|rol|PUBLIC} ...;
```



[Antoine](#) (Open Clip Art .CC)

✓ Del sistema o roles a usuarios:

```
REVOKE {privilegio_stma | rol} [, {privilegio_stma | rol}]...|ALL|PRIVILEGES}
ON [usuario.]objeto
FROM {usuario|rol|PUBLIC} [{usuario|rol|PUBLIC} ...;
```

Juan va a quitar el permiso de seleccionar y de actualizar sobre la tabla Usuarios a Ana:

```
REVOKE SELECT, UPDATE ON Usuarios FROM Ana;
```

y va a quitarle el permiso de eliminar usuarios:

```
REVOKE DROP USER FROM Ana;
```

Autoevaluación

Asocia cada comando con su uso.

Usuarios y permisos.

Comando	Relación	Función
CREATE USER	☐	1. Se utiliza para dar permisos a los usuarios o roles.

Comando	Relación	Función
DROP USER	☐	2. Se utiliza para eliminar usuarios.
GRANT	☐	3. Se utiliza para crear usuarios.
REVOKE	☐	4. Se utiliza para quitar permisos.

Enviar

Estupendo, ya conoces los comandos básicos para la creación de usuarios y sus permisos.

Anexo I.- Elementos del lenguaje SQL.

El lenguaje SQL está compuesto por comandos, cláusulas, operadores, funciones y literales. Todos estos elementos se combinan en las instrucciones y se utilizan para crear, actualizar y manipular bases de datos.

✔ **COMANDOS:**

Comandos DDL. Lenguaje de Definición de Datos:

Comandos DDL. Lenguaje de Definición de Datos.

Comando:	Descripción:
CREATE	Se utiliza para crear nuevas tablas, campos e índices.
DROP	Se utiliza para eliminar tablas e índices.
ALTER	Se utiliza para modificar tablas.

Comandos DML. Lenguaje de Manipulación de Datos:

Comandos DML. Lenguaje de Manipulación de Datos.

Comando:	Descripción:
SELECT	Se utiliza para consultar filas que satisfagan un criterio determinado.
INSERT	Se utiliza para insertar datos en una tabla.
UPDATE	Se utiliza para modificar valores de campos y filas específicos.
DELETE	Se utiliza para eliminar filas de una tabla.

Comandos DCL. Lenguaje de Control de Datos:

Comandos DCL. Lenguaje de Control de Datos.

Comando:	Descripción:
GRANT	Permite dar permisos a uno o varios usuarios o roles para realizar tareas determinadas.
REVOKE	Permite eliminar permisos que previamente se han concedido con GRANT.

✔ **CLÁUSULAS:**

Son palabras especiales que permiten modificar el funcionamiento de un comando.

Cláusulas

Cláusula:	Descripción:
FROM	Especifica la tabla de la que se van a seleccionar las filas.
WHERE	Especifica las condiciones que deben cumplir las filas seleccionadas.
GROUP BY	Permite agrupar las filas seleccionadas según un criterio específico.
HAVING	Especifica condiciones sobre los grupos generados con GROUP BY.
ORDER BY	Ordena los resultados de una consulta.

✔ **OPERADORES:**

Permiten crear expresiones complejas. Pueden ser aritméticos (+, -, *, /, ...) o lógicos (<, >, <=, >=, AND, OR, ...).

Operadores lógicos.

Operador:	Descripción:
AND	Devuelve verdadero si ambas condiciones son ciertas.
OR	Devuelve verdadero si al menos una condición es cierta.
NOT	Invierte el valor de una condición.

✔ **FUNCIONES:**

Para obtener valores complejos. Ejemplo: calcular el promedio de salarios.

Funciones de agregado.

Función:	Descripción:
AVG	Calcula el promedio de los valores de un campo determinado.
COUNT	Devuelve el número de filas seleccionadas.
SUM	Devuelve la suma de todos los valores de un campo determinado.
MAX	Devuelve el valor más alto de un campo determinado.
MIN	Devuelve el valor mínimo de un campo determinado.

✔ **LITERALES:**

Valores concretos, como números, fechas o texto.

Literales

Literal:	Descripción:
'2024-03-14'	Literal de fecha en formato MySQL.

Literal:	Descripción:
'María'	Literal de cadena de caracteres.
5	Literal numérico.

Anexo II.- Instalación de MySQL Server.

En este archivo tienes los pasos para instalar MySQL Server.

[Instalación de MySQL Server](#)

Anexo III. Instalación de MySQL Workbench y primeros pasos

En este archivo tienes información sobre la instalación de MySQL Workbench y los pasos para crear conexiones, crear bases de datos, tablas y otros datos generales. Instálalo en tu ordenador y ejecuta en él todas las sentencias de la unidad para practicar y evitar errores de sintaxis.

[MySQL Workbench: Instalación y primeros pasos](#) (pdf - 964 B)

Anexo IV. Ejercicios DDL con solución

En el siguiente archivo encontrarás enunciados de DDL con sus soluciones, los cuales pueden servirte de ayuda. En informática, no suele haber una única solución, por lo que es recomendable que primero intentes realizarlos y luego compares con la solución propuesta.

[Enunciados de ejercicios DDL y su solución en MySQL](#)

